

一、内存访问与存储优化

1. 内存分配与布局

`compact_buffer_region.cc`: 压缩缓冲区区域, 减少内存碎片。

- **详细解释:** 这个Pass的主要目标是通过分析内存的使用情况, 重新组织缓冲区在内存中的布局, 将原本分散的内存区域进行整合和压缩。这有助于减少内存碎片, 提高内存的局部性, 从而可能减少cache miss, 提升程序整体性能。尤其对于有大量小块内存分配的场景, 效果更明显。
- **实现思路:**
 1. 遍历IR, 收集所有缓冲区的分配信息(大小、使用位置等)。
 2. 分析缓冲区之间的使用关系和生命周期。
 3. 设计一个新的内存布局, 将相关的缓冲区尽可能地放置在一起。
 4. 重写IR中的缓冲区分配和访问指令, 指向新的内存布局。

`merge_shared_memory_allocations.cc`: 合并共享内存分配, 减少开销。

- **详细解释:** 在并行计算(如GPU)中, 共享内存的分配和同步是有一定开销的。此Pass会识别出可以合并的共享内存分配请求, 将多个小的共享内存块合并成一个大的分配, 减少了分配次数和相关的管理开销, 有助于提升并程序效率。
- **实现思路:**
 1. 遍历IR, 识别对共享内存的分配操作。
 2. 分析不同共享内存分配的用途和访问模式。
 3. 如果多个小的分配在同一作用域内且用途兼容, 计算它们总共所需的内存大小。
 4. 将这些小的分配替换为一个大的共享内存分配。
 5. 更新相关的内存访问指令, 使其使用新的大分配内部的偏移量来访问数据。

`storage_access.cc / .h`: 分析和优化存储访问模式。

- **详细解释:** 这组文件包含了用于对程序中的内存访问模式进行静态或动态分析的代码。通过分析数据的读取、写入、访问顺序、步长等信息, 编译器可以更好地理解程序的内存行为, 为后续的存储优化(如数据预取、缓存管理、数据布局转换)提供重要的依据。
- **实现思路:**

1. 遍历IR，识别所有的内存加载（Load）和存储（Store）指令。
2. 分析访问的基地址、索引计算、访问步长、访问频率等信息。
3. 构建数据结构（如访问模式图）来记录和表示这些信息。
4. 提供查询接口，供其他优化Pass获取存储访问模式的分析结果。

storage_rewrite.cc: 重写存储访问模式（如提升共享内存）。

- **详细解释:** 基于存储访问模式的分析结果，此Pass会对内存访问相关的IR（中间表示）进行重写。一个常见的应用是将频繁访问的数据从全局内存（Global Memory）提升到访问速度更快的共享内存（Shared Memory），或者改变访问的顺序以提高缓存命中率和访存合并效率。
- **实现思路:**
 1. 利用 `storage_access.cc` 等工具的分析结果，识别适合进行存储重写的区域（如循环内部对全局内存的频繁访问）。
 2. 决定重写策略（如提升到共享内存）。
 3. 在IR中插入新的共享内存分配。
 4. 在进入计算区域前，插入从全局内存到共享内存的数据拷贝指令。
 5. 修改计算区域内的访存指令，使其访问共享内存。
 6. 在离开计算区域后，如果需要，插入从共享内存到全局内存的数据写回指令。

update_pointer_storage_scope.cc / .h: 更新指针存储作用域（如全局到共享）。

- **详细解释:** 当存储重写Pass改变了数据的存储位置（例如从全局内存移动到共享内存）后，相关的指针或内存访问指令需要更新其指向的存储作用域信息。这组文件负责在IR中正确地标记指针所指向的内存区域类型（如Global、Shared、Local），确保后续的Pass和后端代码生成能够正确处理这些内存访问。
- **实现思路:**
 1. 接收到存储位置变化的通知（例如来自 `storage_rewrite.cc`）。
 2. 遍历IR，找到所有引用到被移动数据的指针或内存访问指令。
 3. 更新这些指针或指令的存储作用域属性，将其标记为新的存储区域（如Shared）。

lower_device_storage_access_info.cc: 降低设备存储访问信息。

- **详细解释:** 这个Pass将设备存储访问的一些高级或抽象的表示降低为更接近硬件或特定后端

的低级表示。这通常是后端代码生成流程中的一个步骤，为生成最终的可执行代码（如 PTX、SPIR-V等）做准备。

- **实现思路:**

1. 遍历IR，查找表示设备存储访问的抽象节点或属性。
2. 根据目标后端的要求，将这些抽象表示转换为具体的加载/存储指令、地址计算逻辑或特定的内置函数调用。
3. 可能需要考虑内存对齐、数据宽度等底层细节。

annotate_device_regions.cc: 标注设备内存区域（如全局/共享/局部）。

- **详细解释:** 此Pass在设备代码的IR中明确地标注不同的内存区域（Global、Shared、Local等）。这有助于编译器和硬件理解数据的存储位置，从而应用针对性的访存优化策略。明确的内存区域信息对于依赖内存层级结构的优化至关重要。

- **实现思路:**

1. 分析IR中缓冲区的分配位置或指针的来源。
2. 根据分配的上下文或指针的属性，确定其所在的内存区域。
3. 在IR中为相应的缓冲区、指针或访存指令添加内存区域的标注（Annotation）。

decorate_device_scope.cc: 修饰设备作用域信息。

- **详细解释:** 这个Pass可能为设备作用域信息添加额外的修饰、属性或元数据。这些附加信息可以用于指导后续的优化Pass或后端代码生成，例如标记某个区域是只读的、易失的等等。

- **实现思路:**

1. 遍历IR中的设备作用域。
2. 根据特定的分析结果或规则，为这些作用域添加额外的修饰或元数据属性。
3. 这些属性可以是只读、易失、特定的缓存控制标记等。

- 2. 特殊内存优化

inject_rolling_buffer.cc: 注入滚动缓冲区优化（减少内存占用）。

- **详细解释:** 滚动缓冲区是一种内存优化技术，通过重用固定大小的缓冲区来处理流式数据。当数据流经过时，缓冲区的内容会不断更新，从而避免为整个数据流分配大块内存。这个Pass将这种模式注入到IR中，通常用于图像处理、视频编码等数据具有时间或空间相关性的应用。

- **实现思路:**

1. 分析循环结构和数据访问模式，识别可以应用滚动缓冲区优化的场景（例如，循环中访问的数据窗口在一个固定大小的缓冲区内滑动）。
2. 计算滚动缓冲区所需的大小。
3. 在IR中创建滚动缓冲区。
4. 修改循环体内的访存指令，使其通过计算在滚动缓冲区内的偏移量来访问数据。
5. 在循环迭代之间，插入更新滚动缓冲区内容的逻辑（例如，通过拷贝或指针移动）。

lower_vtcm_alloc.cc: 处理 VTCM (Video Tightly Coupled Memory) 分配。

- **详细解释:** VTCM (Video Tightly Coupled Memory) 是一种特定硬件（通常是DSP或嵌入式系统）上紧密耦合的内存，具有非常低的访问延迟。这个Pass专门处理针对VTCM的内存分配和管理，将其映射到目标硬件的特定指令或机制。
- **实现思路:**
 1. 识别需要分配在VTCM中的缓冲区或变量（可能通过特定的属性或分析）。
 2. 根据VTCM的特性（大小、访问方式等），在IR中生成对应的VTCM分配指令或表示。
 3. 修改相关的访存指令，使其使用VTCM的访问方式。

memhammer_coalesce.cc: 内存访问合并（提升访存效率）。

- **详细解释:** 在GPU等并行架构上，当多个线程访问相邻的内存地址时，硬件可以将这些独立的访问合并成一个或少数几个更宽的内存事务，这就是内存访问合并。这个Pass会分析并重写访存指令，以便更好地利用硬件的合并能力，显著提高访存带宽和效率。
- **实现思路:**
 1. 遍历IR，分析并行区域内（如GPU的Warp内）线程的内存访问模式。
 2. 如果多个线程访问连续或具有固定跨度的内存地址，识别出可以合并的访问组。
 3. 重写访存指令，将其转换为更宽的加载/存储指令，或者调整访问顺序以促进硬件合并。

memhammer_intermediate_stage.cc: 中间阶段内存优化。

- **详细解释:** 这可能是一系列或某个特定阶段的内存优化Pass，处理一些在其他更通用或更特定的优化Pass之间进行的内存相关的转换或清理工作。

- **实现思路:** (这个是一个通用描述, 具体实现思路取决于它包含的具体优化, 这里提供一个通用的方向)
 1. 根据该阶段需要解决的具体内存问题, 应用相应的分析技术 (如数据流分析、别名分析)。
 2. 基于分析结果, 执行特定的内存相关的IR变换, 例如调整缓冲区大小、插入内存屏障、优化数据移动等。

memhammer_lower_auto_copy.cc: 自动拷贝操作降级。

- **详细解释:** 编译器或某些Pass可能会自动插入内存拷贝操作 (例如在不同内存区域之间移动数据)。这个Pass会将这些自动生成的拷贝操作降低为更具体的底层指令或函数调用, 使其能够被后端正确处理和生成代码。
- **实现思路:**
 1. 遍历IR, 识别由高级Pass自动生成的内存拷贝节点或表示。
 2. 根据源和目标内存区域的类型、数据大小等信息, 将其转换为特定于目标后端的内存拷贝指令 (如GPU的 `memcpy`、DMA操作或一系列加载/存储指令)。

memhammer_tensorcore_rewrite.cc: TensorCore 内存重写规则。

- **详细解释:** TensorCore是NVIDIA GPU中用于加速矩阵乘加等操作的单元。这个Pass应用特定的内存重写规则, 确保数据以适合TensorCore处理的布局 and 方式进行访问, 最大化TensorCore的利用率和性能。
- **实现思路:**
 1. 识别IR中的MMA操作以及相关的输入、权重和输出缓冲区。
 2. 根据TensorCore对输入片段 (fragment) 和数据布局的要求, 分析当前缓冲区的布局是否匹配。
 3. 如果不匹配, 在IR中插入数据重排 (如转置、reshape) 或布局转换操作, 使数据符合TensorCore的要求。
 4. 可能需要调整循环结构和访存模式, 以配合TensorCore的处理粒度。

3. 数据布局与转换

transform_mma_buffer_layout.cc: 转换 MMA (矩阵乘加) 操作的缓冲区布局。

- **详细解释:** 此Pass负责调整用于MMA (Matrix Multiply-Accumulate) 操作的输入、权重和输出缓冲区的内存布局。不同的硬件 (如GPU的TensorCore) 对数据布局有特定的要

求，为了最大化硬件利用率和访存效率，需要将缓冲区数据组织成硬件期望的格式（如行列式布局、交叉存储等）。

- **实现思路:**

1. 识别IR中的MMA操作。
2. 分析输入、权重、输出缓冲区当前的内存布局。
3. 根据目标硬件（如TensorCore）对MMA操作的数据布局要求，确定目标布局。
4. 如果当前布局与目标布局不匹配，在IR中插入数据转换（如转置、reshape、pack/unpack）操作。
5. 修改MMA操作的访存指令，使其使用转换后的数据。

tensorcore_infer_fragment.cc: 推断 TensorCore 片段布局。

- **详细解释:** TensorCore在执行MMA操作时，会将大矩阵分解为小的"片段"（fragment）进行处理。这个Pass会根据MMA操作的属性（如输入矩阵的形状、数据类型等）推断出TensorCore所需的片段布局，为后续的代码生成和内存访问优化提供信息。

- **实现思路:**

1. 识别IR中的TensorCore MMA操作。
2. 根据操作的输入/输出数据类型、形状以及目标TensorCore的特性，计算出每个线程或Warp需要处理的数据片段的大小和布局。
3. 在IR中生成表示片段布局的信息，并与相关的Load/Store指令关联。

inject_permutex_layout.cc: 注入 Permutex 布局（数据重排）。

- **详细解释:** Permutex（Permutation and Extension）是一种数据重排技术，用于改变数据的存储顺序，以适应不同的计算模式或硬件要求。这个Pass将Permutex布局操作注入到IR中，例如用于实现数据的转置、切片或特定模式的重排。

- **实现思路:**

1. 识别需要进行数据重排的缓冲区或计算模式。
2. 根据所需的重排模式，在IR中插入Permutex操作节点。
3. Permutex操作通常涉及到复杂的索引计算和数据移动，Pass需要生成相应的低层IR来实现这些操作。

二、循环变换与并行化

1. 循环优化

unroll_loop.cc: 循环展开。

- **详细解释:** 循环展开是一种经典的优化技术，通过在编译时复制循环体，减少循环的迭代次数和循环控制开销（如分支判断、计数器更新）。这有助于增加指令级并行性，填充流水线，但可能增加代码大小。
- **实现思路:**
 1. 分析循环的迭代次数是否已知或可计算。
 2. 确定展开因子（即每次迭代执行多少次循环体）。
 3. 复制循环体多次，根据展开因子调整循环的边界和步长。
 4. 处理循环的余项（如果总迭代次数不是展开因子的整数倍）。

vectorize_loop.cc: 循环向量化。

- **详细解释:** 循环向量化是将循环中的独立操作转换为向量指令，利用现代处理器中的SIMD（Single Instruction, Multiple Data）单元同时处理多个数据元素。此Pass会分析循环的依赖关系，判断是否可以安全地进行向量化，并生成相应的向量指令。
- **实现思路:**
 1. 分析循环体内的操作以及数据依赖关系。
 2. 判断循环是否可以安全地进行向量化（例如，没有循环携带依赖）。
 3. 确定向量长度（SIMD寄存器的大小）。
 4. 将循环体内的标量操作替换为对应的向量操作。
 5. 可能需要调整内存访问模式，以生成向量加载和存储指令。

split_host_device.cc: 分离主机和设备代码。

- **详细解释:** 在异构计算环境中，程序通常包含在主机（CPU）和设备（如GPU、DSP）上执行的代码。这个Pass将IR中需要在主机和设备上执行的部分分离，生成独立的函数或代码块，为各自的目标平台编译做准备。
- **实现思路:**
 1. 遍历IR，识别标记为在特定设备上执行的代码区域（例如通过Function Attributes或特定的IR节点）。
 2. 将这些设备代码区域提取出来，形成独立的PrimFunc或其他表示。
 3. 在主机代码中，将原设备代码区域替换为设备内核启动的调用。
 4. 为主机和设备代码生成各自的编译和链接流程所需的信息。

loop_partition.cc: 循环分区（如拆分为内/外层）。

- **详细解释:** 循环分区是将一个循环拆分为多个独立的循环或嵌套循环。常见的应用包括将一个循环拆分为内层和外层循环（用于瓦片化/分块），或者将循环的迭代范围分配给不同的线程或处理单元。
- **实现思路:**
 1. 识别需要进行分区的循环。
 2. 确定分区策略和块大小。
 3. 创建新的嵌套循环结构，外部循环遍历块，内部循环处理块内的元素。
 4. 调整循环变量和索引计算，使其正确地访问每个块的数据。
 5. 可能需要配合线程绑定或数据移动等其他优化。

convert_for_loops_serial.cc: 将并行循环转换为串行。

- **详细解释:** 在某些情况下，原本标记为并行执行的循环可能需要转换为串行执行。这可能发生在目标硬件不支持并行、调试目的，或者在某些优化Pass之后并行性不再有利时。此Pass会移除并行相关的标记，将循环转换为普通的串行循环。
- **实现思路:**
 1. 遍历IR，识别标记为并行执行的循环（例如带有线程绑定属性的For循环）。
 2. 移除与并行执行相关的属性或节点。
 3. 将循环转换为标准的串行For循环结构。

2. 并行与线程

inject_virtual_thread.cc: 注入虚拟线程（数据并行）。

- **详细解释:** 虚拟线程是一种在高级IR中表示数据并行的方式，它抽象了底层硬件线程的细节。这个Pass在计算密集型区域注入虚拟线程，标记哪些计算可以并行执行。后续的Pass会将这些虚拟线程映射到目标硬件的实际线程（如CUDA的Thread/Block）。
- **实现思路:**
 1. 分析计算任务的并行性，确定可以在哪些维度进行并行。
 2. 在IR中创建代表虚拟线程的For循环，并为这些循环变量添加线程绑定属性。
 3. 将计算任务的迭代空间映射到这些虚拟线程循环。

remap_thread_axis.cc: 重映射线程轴（优化线程块/网格）。

- **详细解释:** 在并行编程中，线程通常组织成多维的网格和线程块。这个Pass会根据目标硬件

的特性和计算模式，对虚拟线程轴进行重映射和组织，优化线程块和网格的大小、形状，以提高硬件利用率和减少同步开销。

- **实现思路:**

1. 识别带有线程绑定属性的虚拟线程循环。
2. 根据目标硬件的线程组织结构（如GPU的blockIdx.x, threadIdx.y）和性能模型，决定如何将虚拟线程轴映射到实际的硬件线程轴。
3. 重写IR中的线程绑定属性和相关的索引计算。

unify_thread_binding.cc: 统一线程绑定。

- **详细解释:** 在复杂的程序中，不同的计算区域可能采用了不同的线程绑定方式。这个Pass会尝试统一这些线程绑定，使其符合一致的模式，简化后续的优化和代码生成流程。

- **实现思路:**

1. 遍历IR，收集不同计算区域的线程绑定信息。
2. 分析这些绑定方式，确定一个统一或兼容的绑定模式。
3. 重写IR，将不同区域的线程绑定调整为统一的模式，可能需要插入数据重排或同步操作来协调。

lift_thread_binding.cc: 提升线程绑定信息。

- **详细解释:** 这个Pass将线程绑定信息从IR中的局部或内层作用域提升到更高级别，使得编译器能够更全面地了解线程的组织方式，从而应用更广泛的并行优化。

- **实现思路:**

1. 遍历IR，找到在内层作用域定义的线程绑定属性。
2. 分析这些绑定的作用范围和生命周期。
3. 将线程绑定属性移动到更外层的For循环或Function节点上。
4. 确保在提升过程中，线程绑定的语义和正确性得到保持。

lower_thread_allreduce.cc: 降低跨线程规约操作。

- **详细解释:** AllReduce是一种常见的并行通信操作，用于计算所有线程的局部结果的全局和（或其他规约操作）。这个Pass会将抽象的AllReduce操作降低为适合目标硬件的底层通信原语，例如在GPU上使用Warp Shuffle指令进行Warp内规约，或使用共享内存和原子操作进行线程块内规约。

- **实现思路:**

1. 识别IR中的抽象AllReduce操作。

2. 根据目标硬件的特性和规约范围（Warp内、线程块内、网格内），选择合适的底层实现策略。
3. 对于Warp内规约，使用Warp Shuffle指令进行数据交换和计算。
4. 对于线程块内规约，使用Shared Memory暂存部分结果，并结合原子操作或同步原语进行最终规约。

lower_warp_memory.cc: 降低 Warp 级别内存操作（GPU）。

- **详细解释:** 在NVIDIA GPU上，Warp是并行执行的基本单位。这个Pass处理和降低针对Warp级别内存（如Warp Shuffle、Warp-level MMF）的操作，将其转换为底层的PTX指令或其他硬件支持的机制，利用Warp内的高效通信和数据共享。
 - **实现思路:**
 1. 识别IR中标记为Warp级别执行或访问的内存操作。
 2. 将这些操作转换为目标后端（如PTX）支持的Warp级别的内存指令，例如 `__shfl_sync`、`__ldcs` 等。
 3. 可能需要调整数据和索引计算，以符合Warp级别操作的要求。
- ### 三、计算图与表达式优化

1. 表达式简化

simplify.cc / .h: 算术和逻辑表达式简化。

- **详细解释:** 这组文件包含了实现算术和逻辑表达式简化的Pass。这些Pass应用代数规则、常量折叠、死代码消除等技术来简化IR中的表达式，减少不必要的计算，例如将 `(x * 0) + y` 简化为 `y`。
- **实现思路:**
 1. 遍历IR中的表达式树。
 2. 应用预定义的简化规则（例如，`a * 0 = 0`，`a + 0 = a`，`if (true) then x else y = x`）。
 3. 执行常量折叠，计算常量表达式的值。
 4. 识别和移除死代码（计算结果未被使用的表达式）。
 5. 可能需要多次遍历和应用规则，直到IR不再变化。

common_subexpr_elim.cc / .h: 公共子表达式消除 (CSE)。

- **详细解释:** 公共子表达式消除（CSE）是一种识别和消除程序中重复计算相同值的表达式的

优化技术。这个Pass会找到多次出现且计算结果相同的表达式，计算一次后将结果存储在一个临时变量中，后续出现的地方直接使用该临时变量的值，避免了重复计算。

- **实现思路:**

1. 遍历IR，识别计算相同的表达式。这通常需要一个值编号（Value Numbering）或哈希技术来判断表达式的等价性。
2. 记录已经计算过的表达式及其结果（临时变量）。
3. 当遇到一个与之前计算过的表达式相同的表达式时，将其替换为存储结果的临时变量。
4. 确保替换是安全的，即在替换点临时变量的值是有效的。

common_subexpr_elim_tools.cc / .h: CSE 工具函数。

- **详细解释:** 这组文件提供了用于实现CSE Pass的辅助函数和数据结构，例如用于分析表达式依赖关系、管理临时变量等。
- **实现思路:** (提供工具类实现的通用思路)
 1. 实现表达式的哈希或等价性比较函数。
 2. 实现数据结构（如哈希表）来存储表达式和其对应的临时变量。
 3. 实现数据流分析或活跃变量分析，以确保替换的安全性。

hoist_expression.cc: 表达式提升（移出循环）。

- **详细解释:** 表达式提升（Expression Hoisting）是一种将循环体内计算结果不随循环迭代变化的表达式移到循环外部的优化技术。这减少了循环内的计算量，提高了循环的执行效率。
- **实现思路:**
 1. 识别循环结构。
 2. 分析循环体内的表达式，判断其计算结果是否在循环的不同迭代中保持不变（即不依赖于循环变量或循环内修改的变量）。
 3. 将不变的表达式移动到循环之前。
 4. 用提升后的表达式结果替换循环体内原表达式。
 5. 确保提升是安全的，不改变程序的语义。

replace_selected_expr.cc / .h: 替换选定表达式。

- **详细解释:** 这组文件提供了替换IR中特定表达式的功能。这可以用于实现各种优化，例如用更高效的等效表达式替换复杂的表达式，或者在进行某些分析或转换时替换特定的模式。

- **实现思路:**

1. 定义要查找和替换的表达式模式 (Pattern Matching) 。
2. 遍历IR, 查找匹配的表达式。
3. 根据预定义的替换规则, 用新的表达式替换匹配的旧表达式。
4. 确保替换是语义等价的。

2. 分支与选择

reduce_branching_through_overcompute.cc: 通过过量计算减少分支。

- **详细解释:** 过量计算 (Overcompute) 是一种优化技术, 通过在条件分支的两边都执行计算, 然后根据条件选择正确的结果, 来消除分支本身带来的开销 (如分支预测失败的惩罚) 。这个Pass会识别可以应用过量计算的模式, 并重写IR以消除分支。
- **实现思路:**
 1. 识别IR中的条件分支结构 (如IfThenElse) 。
 2. 分析分支两侧 (Then和Else块) 的计算是否可以安全地同时执行 (例如, 没有副作用, 或者副作用可以被处理) 。
 3. 如果可以, 将Then和Else块中的计算合并, 在分支后使用 `select` 操作根据条件选择最终结果。
 4. 删除原有的分支控制流。

rewrite_unsafe_select.cc: 重写不安全的 `select` 操作。

- **详细解释:** `select` 操作通常用于表示条件选择, 例如 `result = condition ? value1 : value2` 。在某些情况下, 如果 `condition` 的计算或者 `value1` 和 `value2` 的计算存在副作用或不确定性, `select` 操作可能是不安全的。这个Pass会识别并重写这些不安全的 `select` 操作, 例如将其转换为更明确的条件分支结构。
- **实现思路:**
 1. 遍历IR, 识别 `select` 操作。
 2. 分析 `condition` 、 `value1` 和 `value2` 的计算是否包含副作用或潜在的运行时错误 (如除以零、访问无效内存) 。
 3. 如果存在不安全性, 将 `select` 操作重写为 IfThenElse 结构, 确保只执行条件满足那一方的计算。

using_assume_to_reduce_branches.cc: 利用 assume 条件减少分支。

- **详细解释:** `assume` 语句是一种向编译器提供程序属性或条件的断言，编译器可以假定这些条件为真进行优化。这个Pass会利用IR中的 `assume` 语句来帮助编译器推断出某些分支条件总是为真或假，从而消除不必要的分支。
- **实现思路:**
 1. 遍历IR，识别 `assume` 语句和条件分支。
 2. 分析 `assume` 语句中的条件，并尝试将其与分支条件关联。
 3. 如果 `assume` 条件能够证明某个分支条件总是为真或假，则将该分支替换为无条件跳转或删除死代码。

3. 常量与内联

extract_constants.cc: 提取常量表达式。

- **详细解释:** 这个Pass会识别程序中的常量表达式（其值在编译时已知）并将其提取出来。这有助于进行常量传播和常量折叠等进一步优化，减少运行时计算。
- **实现思路:**
 1. 遍历IR，识别由常量组成的表达式。
 2. 计算这些常量表达式的值。
 3. 在IR中创建代表这些常量值的新节点（如Constant）。
 4. 用新的Constant节点替换原有的常量表达式。

inline_private_functions.cc: 内联私有函数。

- **详细解释:** 函数内联是将函数调用的代码替换为被调用函数的实际代码。这个Pass专门针对标记为“私有”的函数进行内联，消除函数调用开销，增加指令级并行性，并可能暴露更多的跨过程优化机会。
- **实现思路:**
 1. 遍历IR，识别对私有函数的调用点。
 2. 复制被调用函数的函数体。
 3. 在调用点，用被调用函数的复制体替换函数调用指令。
 4. 处理参数传递和返回值，确保语义正确。
 5. 移除不再被调用的私有函数定义。

四、硬件特化与低级优化

1. GPU 优化

default_gpu_schedule.cc: 默认 GPU 调度策略。

- **详细解释:** 这个文件定义了TVM在没有用户提供具体调度脚本时，针对GPU设备应用的默认调度策略。这些策略通常基于对常见深度学习算子的分析和对GPU硬件特性的了解，旨在提供一个合理的性能基线。它可能包括线程块/网格大小的选择、循环的自动瓦片化和并行化等。
- **实现思路:**
 1. 分析输入的计算图结构（如卷积、矩阵乘法）。
 2. 根据算子类型和形状，查询预定义的硬件性能模型或启发式规则库。
 3. 基于规则和模型，自动决定循环的并行化策略（如哪些维度绑定到blockIdx，哪些绑定到threadIdx）、瓦片大小、共享内存的使用等。
 4. 将这些调度决策应用到IR中，重写循环和访存结构。

inject_ptx_async_copy.cc: 注入 PTX 异步拷贝指令。

- **详细解释:** 在NVIDIA GPU上，PTX（Parallel Thread Execution）是中间汇编语言。异步拷贝（Async Copy）允许数据在计算进行的同时在内存（如Global Memory到Shared Memory）之间传输，从而隐藏访存延迟。这个Pass会在IR中注入代表PTX异步拷贝操作的指令，以利用这一硬件特性。
- **实现思路:**
 1. 识别需要从全局内存加载到共享内存的数据块，以及使用这些数据的数据区域。
 2. 在计算区域开始之前，插入异步拷贝操作，请求数据从全局内存传输到共享内存。
 3. 在使用数据之前，插入同步指令（如 `cuda::pipeline_commit_and_wait`），确保异步拷贝完成。
 4. 修改计算区域内的访存指令，使其访问共享内存。

inject_ptx_ldg32.cc: 注入 PTX LDG (缓存加载) 指令。

- **详细解释:** LDG（Load Global）是PTX中一种用于从全局内存加载数据的指令，它通常会经过纹理缓存或只读缓存，对于某些访问模式（如广播）可能比普通的加载指令更高效。这个Pass会在合适的场景注入PTX LDG指令。
- **实现思路:**
 1. 遍历IR，识别从全局内存加载数据的操作（Load）。
 2. 分析加载操作的访问模式（如是否是广播访问）。
 3. 如果访问模式适合使用LDG（例如，多个线程加载同一地址或相邻地址），则将普通的Load指令替换为PTX LDG内置函数或表示。

lower_cross_thread_reduction.cc: 降低跨线程规约操作。

- **详细解释:** 跨线程规约（如求和、最大值）是在多个线程的结果上计算一个单一结果的操作。在GPU上，这需要线程间的通信和同步。这个Pass会将高级的规约操作降低为使用底层硬件原语（如Shared Memory、原子操作、Warp Shuffle指令）实现的低效版本。
- **实现思路:**
 1. 识别IR中的规约操作（如Reduce）。
 2. 根据规约的范围（Warp内、线程块内），选择相应的底层实现策略。
 3. 对于Warp内规约，使用Warp Shuffle指令进行数据交换和计算。
 4. 对于线程块内规约，使用Shared Memory暂存部分结果，并结合原子操作或同步原语进行最终规约。

lower_device_kernel_launch.cc: 降低设备内核启动。

- **详细解释:** 这个Pass处理将计算任务作为设备内核在目标硬件上启动的操作。它会将抽象的内核启动表示转换为特定于目标平台的API调用，例如对于CUDA生成 `cudaLaunchKernel` 调用。
- **实现思路:**
 1. 识别IR中表示设备内核的代码块（PrimFunc）。
 2. 生成调用目标平台运行时API（如CUDA Runtime API）的代码，用于配置和启动内核。
 3. 包括设置网格和线程块维度、传递内核参数、处理异步执行等。

2. 数据类型与精度

dtype_conversion.cc / .h: 数据类型转换（如 float16）。

- **详细解释:** 这组文件实现了在不同数据类型之间进行转换的功能，例如将计算从float32转换为float16或int8，以利用硬件对低精度计算的支持，减少内存带宽需求和计算量，但可能牺牲精度。
- **实现思路:**
 1. 遍历IR，识别可以进行数据类型转换的操作（例如，将float32的乘加转换为float16）。
 2. 分析转换的安全性（是否会导致精度损失、溢出等）。
 3. 如果安全且有益于性能，在IR中插入数据类型转换（Cast）操作。
 4. 修改相关的计算操作，使其使用新的数据类型。

narrow_datatype.cc: 缩窄数据类型 (如 int64 -> int32)。

- **详细解释:** 当分析确定一个变量或表达式的取值范围可以由一个更窄的数据类型表示时 (例如int64变量的实际取值总是在int32范围内), 这个Pass会将其数据类型缩窄, 从而节省内存和寄存器资源。
- **实现思路:**
 1. 对变量或表达式进行值范围分析。
 2. 如果分析确定其取值范围可以完全由一个更窄的数据类型表示, 则进行类型缩窄。
 3. 在IR中修改变量或表达式的数据类型。
 4. 可能需要插入类型转换以确保操作的正确性。

force_narrow_index_to_i32.cc: 强制索引类型为 int32。

- **详细解释:** 在某些硬件平台或API中, 内存访问的索引必须是32位整数。这个Pass会强制将所有用于内存访问的索引的数据类型转换为int32, 以确保兼容性和正确性。
- **实现思路:**
 1. 遍历IR, 识别用于数组或缓冲区访问的索引表达式。
 2. 如果索引的数据类型不是int32, 在其前面插入一个 Cast 操作, 将其转换为int32。
 3. 需要确保原始索引的值在int32的范围内, 否则转换可能不安全。

lower_custom_datatypes.cc: 降低自定义数据类型操作。

- **详细解释:** TVM支持自定义数据类型。这个Pass会将这些自定义数据类型上的操作降低为使用内置数据类型的操作序列, 以便后端能够处理。
- **实现思路:** (与 lower_intrin 类似)
 1. 遍历IR, 识别使用自定义数据类型的操作 (如算术运算、加载、存储)。
 2. 根据自定义数据类型的定义和后端支持的内置类型, 将这些操作分解或转换为使用内置数据类型的等效操作序列。
 3. 可能需要插入位操作、类型转换等。

unsupported_dtype_legalize.cc: 不支持数据类型的合法化。

- **详细解释:** 如果程序中使用了目标硬件或后端不支持的数据类型, 这个Pass会尝试将这些操作"合法化", 例如通过模拟或分解操作, 将其转换为可以使用支持的数据类型实现的形式。
- **实现思路:**

1. 遍历IR，识别目标后端不支持的数据类型及其上的操作。
2. 根据预定义的合法化规则，将不支持的操作替换为使用支持数据类型和操作实现的等效计算图。
3. 例如，用一系列支持的算术和位操作来模拟浮点运算。

3. Pipeline 与缓冲

inject_double_buffer.cc: 注入双缓冲优化。

- **详细解释:** 双缓冲是一种通过使用两块交替使用的缓冲区来隐藏数据加载/存储延迟的技术。当一个缓冲区用于计算时，另一个缓冲区同时进行数据的加载或存储。这个Pass会将双缓冲机制注入到IR中，例如用于优化计算密集型循环的数据访存。
- **实现思路:**
 1. 识别可以进行双缓冲的循环结构，其中数据加载/存储和计算可以重叠。
 2. 为需要双缓冲的数据创建两组缓冲区（"前台"和"后台"）。
 3. 修改循环结构：在每个迭代中，使用一个缓冲区进行计算，同时在后台加载/存储下一个迭代所需的数据到另一个缓冲区。
 4. 插入同步指令，确保在访问数据前加载已完成，在写回数据前计算已完成。

inject_software_pipeline.cc: 注入软件流水线。

- **详细解释:** 软件流水线是一种编译器技术，通过重排指令顺序，使得来自不同循环迭代的指令可以重叠执行，从而提高循环的吞吐量。这个Pass会分析循环的依赖关系，并重排IR中的指令以实现软件流水线。
- **实现思路:**
 1. 分析循环体内的指令及其依赖关系。
 2. 确定流水线的阶段和每个阶段的指令。
 3. 重排循环体内的指令，使得来自后续迭代的指令可以在当前迭代的较早阶段开始执行。
 4. 可能需要引入新的寄存器或临时变量来保存跨迭代的值。
 5. 处理循环的 prolog（开始阶段）和 epilog（结束阶段），填充和排空流水线。

五、IR 变换与降级

1. IR 工具与辅助

ir_utils.cc / .h: IR 实用工具函数。

- **详细解释:** 这组文件包含了一系列用于操作和分析TVM中间表示 (IR) 的实用工具函数。这些函数提供了对IR结构进行遍历、查询、修改等基本操作的能力，是实现各种IR Pass的基础。
- **实现思路:** (提供工具类实现的通用思路)
 1. 实现IR节点的访问者模式 (Visitor Pattern) ，用于遍历IR树。
 2. 实现IR节点的克隆、比较、替换等基本操作。
 3. 提供用于查询IR节点属性、父节点、子节点等的函数。
 4. 可能包含用于构建常见IR结构的辅助函数。

primfunc_utils.cc: PrimFunc 实用工具。

- **详细解释:** PrimFunc是TVM中用于表示计算函数的核心IR结构。这个文件提供了一系列专门用于操作和分析PrimFunc的工具函数，例如获取函数的输入/输出参数、遍历函数体、查找特定的IR节点等。
- **实现思路:** (提供工具类实现的通用思路)
 1. 提供用于访问PrimFunc的属性，如函数名、参数列表、返回值类型、函数体等。
 2. 实现用于遍历PrimFunc函数体 (如Basic Block、Control Flow) 的工具。
 3. 提供用于在函数体中查找特定类型IR节点 (如Load, Store, Call) 的函数。
 4. 可能包含用于修改PrimFunc结构 (如添加/删除参数、插入/删除语句) 的函数。

arg_binder.cc / .h: 参数绑定器。

- **详细解释:** 参数绑定器用于将函数的抽象参数绑定到具体的存储位置或值。在编译过程中，这一步是将函数的逻辑描述映射到实际的内存分配和寄存器分配的基础。
- **实现思路:**
 1. 接收函数参数和对应的绑定目标 (如内存地址、常量值、寄存器) 。
 2. 构建一个映射表，记录参数到绑定目标的对应关系。
 3. 在IR遍历过程中，当遇到参数的使用时，根据映射表将其替换为绑定的值或地址。

bind_params.cc: 绑定参数到常量。

- **详细解释:** 这个Pass专门负责将函数的参数绑定到常量值，这通常发生在模型推理阶段，模型的权重和偏置等参数是固定的常量。将参数绑定为常量有助于编译器进行常量传播和折叠等进一步优化。

- **实现思路:**

1. 接收需要绑定的参数列表和对应的常量值。
2. 遍历IR，找到对这些参数的使用。
3. 将参数的使用替换为对应的常量值。
4. 可能需要进行常量传播和死代码消除来清理替换后的IR。

2. IR 结构变换

convert_blocks_to_opaque.cc: 将块转换为不透明块。

- **详细解释:** 这个Pass将IR中的特定计算块标记为"不透明"。不透明块内部的细节对外部Pass是隐藏的，这可以用于模块化、保护敏感信息，或者防止某些优化跨越块边界，保持块的封装性。
- **实现思路:**
 1. 识别需要转换为不透明的IR块（例如，通过特定的属性或用户指定）。
 2. 创建一个代表不透明块的新IR节点。
 3. 将原块内部的IR结构作为元数据或属性存储在新节点中。
 4. 在原块的位置替换为不透明块节点。

lower_opaque_block.cc: 降低不透明块。

- **详细解释:** 与 `convert_blocks_to_opaque.cc` 相反，这个Pass会将不透明块降低，暴露其内部的IR结构，使得后续的Pass可以对块内部进行进一步的分析和优化。
- **实现思路:**
 1. 遍历IR，识别不透明块节点。
 2. 从不透明块节点中提取出其内部存储的原始IR结构。
 3. 在不透明块的位置替换为提取出的原始IR结构。

lower_match_buffer.cc: 降低 match_buffer 操作。

- **详细解释:** `match_buffer` 是TVM中用于描述计算操作的输入/输出缓冲区与计算逻辑之间如何关联的机制。这个Pass会将抽象的 `match_buffer` 操作转换为更底层的内存访问和索引计算表示。
- **实现思路:**
 1. 遍历IR，识别 `match_buffer` 节点。
 2. 分析 `match_buffer` 描述的输入/输出缓冲区和计算区域的关系。

3. 根据这些关系，生成具体的内存加载/存储指令、地址计算逻辑、以及循环结构。
4. 用生成的低层IR替换 `match_buffer` 节点。

lower_init_block.cc: 降低初始化块。

- **详细解释:** 初始化块用于描述缓冲区或其他状态的初始化过程。这个Pass会将初始化块的抽象描述转换为具体的底层代码，例如内存清零或填充特定值。
- **实现思路:**
 1. 遍历IR，识别初始化块节点。
 2. 分析初始化块描述的初始化内容（例如，将缓冲区清零，或者用特定值填充）。
 3. 根据初始化内容，生成相应的低层IR，例如循环和Store指令用于填充，或者调用内存设置函数。
 4. 用生成的低层IR替换初始化块节点。

lower_intrin.cc: 降低内建函数操作。

- **详细解释:** 内建函数（Intrinsic Function）是编译器识别并能用特定底层指令或短序列代码替换的函数（例如数学函数sin、cos，或原子操作）。这个Pass会将IR中的内建函数调用降低为目标硬件支持的实际指令或函数调用。
- **实现思路:**
 1. 遍历IR，识别内建函数调用节点。
 2. 根据内建函数的类型和目标后端，查询对应的底层实现方式（可能是特定的汇编指令、库函数调用或一段IR序列）。
 3. 用底层实现替换内建函数调用节点。

lower_tvm_builtin.cc: 降低 TVM 内建操作。

- **详细解释:** 这个Pass处理TVM自身定义的一些内建操作，将其转换为更底层的IR或目标后端支持的表示。
- **实现思路:** (与 lower_intrin 类似)
 1. 遍历IR，识别TVM内建操作节点。
 2. 根据内建操作的类型和目标后端，查询对应的底层实现方式。
 3. 用底层实现替换内建操作节点。

renew_defs.cc: 更新定义。

- **详细解释:** 在IR变换过程中，变量和表达式的定义可能会发生变化。这个Pass会遍历IR，确保所有使用到的变量和表达式都引用到正确的、最新的定义。
- **实现思路:**
 1. 遍历IR，跟踪变量和表达式的定义和使用关系。
 2. 当IR变换导致定义变化时，更新所有引用该定义的使用点，使其指向新的定义。
 3. 可以使用符号表或数据流分析来管理定义和使用之间的关系。

renormalize_split_pattern.cc: 重新规范化拆分模式。

- **详细解释:** 在进行循环拆分（Loop Splitting）等变换后，IR中表示拆分的方式可能需要进行规范化处理，以确保一致性和便于后续Pass的处理。这个Pass负责重新组织和规范化这些拆分模式。
- **实现思路:**
 1. 识别由循环拆分等变换产生的非规范化IR结构。
 2. 应用预定义的规范化规则，将这些结构转换为标准形式（例如，统一循环变量的命名、调整循环边界和步长）。
- 3. API 处理

make_packed_api.cc: 生成 PackedFunc API。

- **详细解释:** PackedFunc是TVM中一种灵活的、可动态调用的函数接口格式。这个Pass负责将内部的PrimFunc转换为外部可以调用的PackedFunc API，包括参数的序列化/反序列化、类型检查等逻辑。
- **实现思路:**
 1. 接收PrimFunc作为输入。
 2. 生成一个新的PackedFunc结构，包含参数列表、属性等。
 3. 生成处理参数序列化/反序列化、类型检查、错误处理的代码。
 4. 生成调用内部PrimFunc的代码，并处理返回值。

make_unpacked_api.cc: 生成 Unpacked API。

- **详细解释:** Unpacked API是另一种形式的函数接口，通常更接近传统的函数调用约定。这个Pass负责生成这种格式的API。
- **实现思路:**

1. 接收PrimFunc作为输入。
2. 生成一个新的函数，其签名符合传统的函数调用约定（例如，参数直接通过寄存器或栈传递）。
3. 在生成的函数体中，直接调用内部的PrimFunc，处理参数和返回值。

六、代码清理与规范

1. 冗余消除

remove_no_op.cc / .h: 删除无操作 (No-Op) 语句。

- **详细解释:** 无操作 (No-Op) 语句是指那些执行后没有任何可见效果的语句。这个Pass会识别并删除这些冗余的No-Op语句，减小代码大小，提高执行效率。
- **实现思路:**
 1. 遍历IR，识别代表No-Op的节点或语句。
 2. 从IR序列中移除这些节点或语句。
 3. 需要确保删除No-Op不会改变控制流或数据依赖。

remove_store_undef.cc: 删除对未定义值的存储。

- **详细解释:** 这个Pass会识别并删除那些将结果存储到未定义变量或内存区域的操作，这种操作通常是程序中的错误或冗余计算。
- **实现思路:**
 1. 对变量进行定义和使用分析，识别哪些变量在使用前没有被定义。
 2. 遍历IR，找到将值存储到未定义变量的Store操作。
 3. 删除这些冗余的Store操作。

remove_weight_layout_rewrite_block.cc: 删除权重布局重写块。

- **详细解释:** 在某些模型或优化流程中，可能存在用于调整权重布局的特定代码块。如果这些重写操作在前面的Pass中已经完成或不再需要，这个Pass会删除这些冗余的块。
- **实现思路:**
 1. 识别IR中标记为权重布局重写或与特定权重转换相关的代码块。
 2. 分析这些块是否仍然必要（例如，检查权重是否已经处于目标布局）。
 3. 如果不需要，删除这些代码块。

remove_assume.cc: 移除 assume 语句（条件已满足）。

- **详细解释:** `assume` 语句是向编译器提供的关于程序状态的断言。如果通过静态分析或其他方式可以确定 `assume` 中的条件总是满足，那么这个Pass会移除该 `assume` 语句，因为它不再为编译器提供新的信息。
- **实现思路:**
 1. 遍历IR，识别 `assume` 语句。
 2. 对 `assume` 条件进行静态分析或利用之前的分析结果。
 3. 如果条件被证明总是为真，则删除该 `assume` 语句。

skip_assert.cc: 跳过 assert 语句（生产环境）。

- **详细解释:** `assert` 语句用于在运行时检查程序状态，如果不满足条件则终止程序。在生产环境中，为了性能考虑，通常会跳过或禁用断言。这个Pass会移除或标记 `assert` 语句，使其在特定编译配置下被忽略。
- **实现思路:**
 1. 遍历IR，识别 `assert` 语句。
 2. 根据编译配置（例如，是否是Release模式），决定是删除 `assert` 语句，还是将其转换为一个条件分支，在生产模式下不执行断言检查。

2. 边界检查

bound_checker.cc: 边界检查插入或优化。

- **详细解释:** 这个Pass负责处理内存访问（如数组索引）的边界检查。它可以选择在编译后的代码中插入运行时边界检查，以防止越界访问，或者在可能的情况下通过静态分析优化或消除不必要的边界检查。
- **实现思路:**
 1. 遍历IR，识别数组或缓冲区访问操作（Load, Store）。
 2. 对于每个访问操作，分析其索引表达式和缓冲区的边界。
 3. **插入检查:** 如果无法在编译时确定访问总是在边界内，在访问前插入运行时条件检查，如果越界则触发错误。
 4. **优化检查:** 利用静态分析（如值范围分析）来证明某些访问总是在边界内，然后删除不必要的运行时检查。

七、分析与调试

profile_instrumentation.cc: 性能分析插桩。

- **详细解释:** 这个Pass会在生成的代码中插入用于性能分析的"插桩" (Instrumentation) 代码。这些插桩代码可以在程序运行时收集性能数据，例如函数的执行时间、内存访问次数等，用于性能调优和瓶颈分析。
- **实现思路:**
 1. 识别需要在哪些地方进行性能分析（例如，函数入口/出口、循环、重要的计算块）。
 2. 在这些位置插入特定的IR节点或函数调用，用于记录时间戳、计数器或其他性能指标。
 3. 这些插桩代码通常会调用一个运行时库来收集和报告数据。

manifest_shared_memory_local_stage.cc: 显式共享内存局部阶段。

- **详细解释:** 这个Pass可能用于在IR中更明确地表示数据在共享内存中的局部阶段。这有助于编译器理解数据在共享内存中的生命周期和使用模式，从而进行更精细的内存管理和同步优化。
- **实现思路:**
 1. 识别使用共享内存的计算模式，特别是在循环内部。
 2. 在IR中显式地表示数据从全局内存加载到共享内存、在共享内存中进行计算、以及可能写回全局内存的各个阶段。
 3. 这可能涉及到插入Shared Memory分配、数据拷贝指令、以及同步屏障。

八、其他

combine_context_call.cc: 合并上下文调用。

- **详细解释:** 这个Pass尝试合并程序中对同一上下文 (Context) 的多次调用，减少函数调用开销。
- **实现思路:**
 1. 遍历IR，识别对同一上下文对象的多次创建或访问。
 2. 如果这些调用可以在不改变语义的情况下合并，则将其替换为一次调用，并在后续使用中引用同一个上下文对象。

replace_global_vars.cc: 替换全局变量。

- **详细解释:** 这个Pass用于替换程序中的全局变量，可能将其转换为局部变量、函数参数，或将其存储在特定的内存区域，以便更好地管理和优化。

- **实现思路:**

1. 识别IR中的全局变量引用。
2. 根据替换策略，在IR中创建新的局部变量、函数参数，或在特定内存区域分配空间。
3. 将全局变量的引用替换为对新创建的变量或内存位置的访问。
4. 需要处理全局变量的初始化和生命周期。

legalize_packed_calls.cc: 合法化 PackedFunc 调用。

- **详细解释:** PackedFunc调用是一种通用的函数调用机制，但其实现可能因目标平台而异。这个Pass会将PackedFunc的调用转换为目标后端支持的、更具体的底层调用方式。

- **实现思路:**

1. 遍历IR，识别PackedFunc调用节点。
2. 根据目标后端的ABI（Application Binary Interface）和PackedFunc的实现细节，生成将参数打包、调用实际函数、解包返回值的底层代码。
3. 这可能涉及到使用特定的寄存器、栈布局或内置函数。

计算图优化

DeadCodeElimination: 删除不影响程序结果的 let 绑定表达式。

- **详细解释:** 这个 Pass 会识别并移除程序中那些计算结果没有被任何后续部分使用的 `let` 绑定表达式（即死代码）。如果 `inline_once` 设置为 `true`，对于那些只被引用了一次的 `let` 绑定，其右侧的表达式会被直接内联到使用点，然后移除 `let` 绑定本身，从而减少变量引入和可能的开销。这个优化有助于减小代码体积，提高执行效率。

- **实现思路:**

1. 遍历 IR，构建一个使用计数（Use Count）或引用图。
2. 识别使用计数为零的 `let` 绑定。
3. 移除使用计数为零的 `let` 绑定及其右侧的表达式。
4. 如果 `inline_once` 为 `true`，识别使用计数为一的 `let` 绑定。
5. 将使用计数为一的 `let` 绑定右侧表达式复制到唯一的使用点。
6. 移除使用计数为一的 `let` 绑定。
7. 需要注意处理带有副作用的表达式，默认情况下不会移除或内联它们，除非 `ignore_purity` 设置为 `true`。

LazyGradientInit: 将 TensorType 的表达式转换为 GradCell。

- **详细解释:** 这个 Pass 主要用于自动微分（如反向模式 AD）场景，它将表示张量的 `TensorType` 表达式转换为 `GradCell` 类型。`GradCell` 是一种代数数据类型，可以延迟或减少内存分配。例如，对于 `ones`，`ones_like`，`zeros`，`zeros_like` 等操作，它们不会立即在内存中实例化一个张量，而是在真正需要时才进行。`GradCell` 还定义了适用于梯度计算的 `+` 和 `*` 操作，这对于处理零填充或一填充张量特别有用，可以提升性能。
- **实现思路:**
 1. 遍历 IR，识别 `TensorType` 类型的表达式，特别是 `ones`，`zeros` 等构造张量的操作。
 2. 将这些表达式转换为 `GradCell` 类型的表示。
 3. 重载或特殊处理 `GradCell` 类型的运算操作（如加法、乘法），使其具备延迟计算或利用稀疏特性的能力。
 4. 修改相关的函数签名和类型标注，反映 `GradCell` 的使用。

FoldConstant: 折叠常量表达式。

- **详细解释:** 此 Pass 会识别程序中所有由常量组成的表达式，并在编译时计算出它们的值，然后用计算出的常量值替换原有的表达式。这消除了运行时的冗余计算，例如将 `(3 + 5) * 2` 直接计算为 `16`。默认情况下，为了兼容性会跳过 QNN 原语的常量折叠，除非特别指定 `fold_qnn` 为 true。
- **实现思路:**
 1. 遍历 IR，识别所有常量节点和只包含常量输入的表达式。
 2. 对这些表达式进行求值。
 3. 用计算出的常量值创建新的 `Constant` 节点。
 4. 用新的 `Constant` 节点替换原有的常量表达式。
 5. 需要处理不同数据类型的常量折叠，并考虑可能的溢出。

SplitArgs: 将参数数量过多的函数拆分成更小的函数。

- **详细解释:** 当一个函数的参数数量非常巨大时，可能会给编译器后端、调用约定或某些分析带来困难。这个 Pass 会将这类参数过多的函数进行拆分，生成一个或多个新的、参数数量较少的辅助函数，并通过结构体或元组等方式在这些函数之间传递参数，从而降低函数的接口复杂度。
- **实现思路:**
 1. 遍历 IR，识别参数数量超过指定阈值 `max_function_args` 的函数。

2. 根据参数分组策略，创建新的函数签名和函数体。
3. 将原函数的参数分配到新的函数中，可能需要创建结构体或元组来打包参数。
4. 修改原函数的调用点，使其调用新生成的函数序列，并处理参数的传递和结果的返回。
5. 需要确保拆分过程不改变程序的语义。

FuseOps: 将操作融合到表达式中，形成独立的函数。

- **详细解释:** 操作融合是一种重要的图优化技术，它将计算图中的多个连续操作（如卷积后接偏置加法、再接激活函数）合并成一个大的"融合"操作。这样做可以减少中间数据的存取（尤其是在寄存器或缓存中），降低内核启动开销，提高计算密集型任务的效率。这个 Pass 会分析计算图的依赖关系和操作的兼容性，识别可融合的模式，并生成代表融合操作的新函数。
- **实现思路:**
 1. 分析计算图的连接关系，识别可以串联执行的操作序列。
 2. 根据预定义的融合规则和策略（例如，哪些操作可以融合，融合的粒度），确定融合边界。
 3. 将被融合的操作提取出来，生成一个新的 PrimFunc 或其他表示。
 4. 在原图中，用一个对新融合函数的调用替换被融合的操作序列。
 5. 处理输入和输出的映射关系。

DefuseOps: FuseOps 的逆操作，将融合后的程序恢复到融合前的状态。

- **详细解释:** 这个 Pass 是 FuseOps 的逆过程。它会将经过 FuseOps 融合后的计算图或函数，拆解回融合之前的原始操作序列。这在某些场景下是必需的，例如在进行一些需要暴露原始操作细节的分析或变换之前，或者为了适配不支持特定融合模式的后端。
- **实现思路:**
 1. 遍历 IR，识别由 FuseOps 生成的融合操作节点（通常是对特定融合函数的调用）。
 2. 从融合操作节点中获取其内部包含的原始操作信息（这需要在融合过程中保留这些信息）。
 3. 在原图中，用提取出的原始操作序列替换融合操作节点。
 4. 恢复原始操作之间的连接和数据依赖关系。

RewriteAnnotatedOps: 重写带有注解的程序。

- **详细解释:** 此 Pass 根据计算图中操作 (Ops) 上的注解 (Annotation) 来重写程序。注解通常用于标记操作应该在何种设备上执行 (例如 CPU、GPU)、使用哪种特定实现, 或者应用特定的优化策略。这个 Pass 会读取这些注解, 并据此修改 IR, 例如插入设备切换操作、调用特定后端的实现函数等。 `fallback_device` 参数指定了没有明确注解的操作默认在何种设备上执行。
- **实现思路:**
 1. 遍历 IR, 检查 Call 节点上的注解属性。
 2. 根据注解指定的目标设备或特定属性, 查找对应的重写规则或目标实现。
 3. 根据规则, 修改 Call 节点, 例如替换为调用目标设备的算子实现, 或者在 Call 节点前后插入设备切换或数据拷贝操作。
 4. 对于没有注解的操作, 使用 `fallback_device` 指定的设备进行处理。

ToBasicBlockNormalForm: 将表达式转换为基本块范式 (Basic Block Normal Form)。

- **详细解释:** 基本块范式是一种 IR 表示形式, 它将程序组织成基本块 (Basic Block) 的序列。每个基本块是一系列顺序执行的指令, 只有一个入口和一个出口。这种范式有助于控制流分析和许多依赖于基本块结构的优化。在此范式下, 每个图节点只属于一个基本块, 跨基本块使用的值通过在共同祖先块中定义的 Var 来引用。
- **实现思路:**
 1. 遍历 IR, 识别控制流结构 (如 If、Loop) 。
 2. 根据控制流划分基本块。
 3. 确保每个基本块内部是线性的指令序列。
 4. 识别跨基本块使用的变量或表达式。
 5. 在这些变量/表达式的最小公共祖先基本块中, 为其创建 Var 绑定。
 6. 修改使用点, 使其引用对应的 Var。

ToANormalForm: 将数据流图转换为 A-Normal Form (ANF)。

- **详细解释:** A-Normal Form 是一种中间表示形式, 它使得所有表达式的参数都是简单的变量或常量。复杂的嵌套表达式会被分解, 并将中间结果绑定到 `let` 变量中。这使得数据流更加显式, 便于数据流分析和后续的 Pass 处理。在此范式下, 具有共享的表达式会被明确地通过 `let` 绑定来表示共享。
- **实现思路:**

1. 遍历 IR 表达式树。
2. 识别作为其他表达式参数的复杂表达式（非 Var 非 Constant）。
3. 为这些复杂表达式创建 `let` 绑定，并生成新的 Var。
4. 将原参数位置的复杂表达式替换为新生成的 Var。
5. 确保每个表达式的计算结果都被绑定到一个 `let` 变量。
6. 维护表达式的顺序（通常是后序遍历）。

ToCPS: 将表达式转换为 Continuation Passing Style (CPS)。

- **详细解释:** Continuation Passing Style (CPS) 是一种编程语言转换技术，它将函数调用和控制流显式化。在 CPS 中，函数不直接返回值，而是接受一个额外的参数，即"continuation"（延续），并将结果传递给这个 continuation 函数。所有计算都会被包装起来并调用 continuation。这种风格可以用来显式化控制流，便于实现特定的优化或转换（例如尾调用优化）。
- **实现思路:**
 1. 遍历 IR 中的函数定义和函数调用。
 2. 修改函数签名，增加一个 continuation 参数（通常是一个函数类型）。
 3. 修改函数体，将原有的返回值计算替换为调用 continuation 并传入计算结果。
 4. 修改函数调用点，构建一个 continuation 函数（通常是 Lambda 或 LetFn），该 continuation 包含了调用点之后的计算，并将这个 continuation 作为参数传递给被调函数。

ToGraphNormalForm: 移除 let 绑定，通过指针直接共享。

- **详细解释:** 这个 Pass 会移除 IR 中的所有 `let` 绑定。原本通过 `let` 变量共享的表达式结果，会被转换为通过直接的指针引用来表示共享。这种形式更接近于底层的图表示，可能用于某些特定的后端或分析。
- **实现思路:**
 1. 遍历 IR，识别 `let` 绑定。
 2. 对于每个 `let` 绑定，识别其右侧的表达式和所有使用绑定变量的地方。
 3. 在使用点，用指向原表达式（或其复制，取决于共享语义）的直接引用替换绑定变量。
 4. 移除 `let` 绑定节点。
 5. 需要谨慎处理引用计数和内存管理，确保正确地共享和释放资源。

PartialEval: 激进的常量传播/常量折叠/内联。

- **详细解释:** 这个 Pass 执行激进的编译时计算，包括常量传播、常量折叠和函数内联。它会尽可能多地在编译阶段计算表达式的值，将常量值传播到使用点，并内联函数调用。目标是最大化编译时计算，减少运行时开销，并为后续的优化（特别是操作融合）创造更多机会。然而，这可能会显著增加代码大小。
- **实现思路:**
 1. 结合多种表达式简化技术（如算术简化、逻辑简化、常量折叠）。
 2. 反复遍历 IR，直到不再发生变化。
 3. 对于常量表达式，计算其值并替换。
 3. 将常量值传播到使用该常量的表达式中，可能引发新的常量折叠机会。
 4. 对于满足内联条件的函数调用，用被调函数的函数体替换调用点。
 5. 需要处理递归、函数副作用等复杂情况。

SimplifyInference: 简化推理过程中的某些操作。

- **详细解释:** 此 Pass 专门针对模型推理阶段进行优化。它会识别并简化一些在推理时具有特定模式的操作，例如，对于一个批归一化（Batch Norm）操作，如果在推理时只需要其输出的第一个分量（通常是归一化后的结果），Pass 会将其展开或重写为一系列更简单的操作（如减法、乘法、加法），而不是保留完整的 Batch Norm 结构，从而减少计算和内存开销。
- **实现思路:**
 1. 遍历 IR，识别推理模式下具有可简化模式的操作（例如，Batch Norm 后接 GetItem(0)）。
 2. 分析操作的输入、属性和上下文，判断是否满足简化条件。
 3. 如果满足，用一系列等价的、更简单的基本操作（如 Cast、Add、Multiply、Subtract、Divide、Reshape 等）替换原有的复杂操作。
 4. 需要处理不同操作类型的简化规则。

FastMath: 用快速但近似的对应项替换非线性激活函数。

- **详细解释:** 这个 Pass 会将计算图中某些非线性激活函数（如 Sigmoid、Tanh）替换为它们更快速但可能精度较低的近似实现。这通常用于对性能要求极高但对精度要求相对宽松的场景，例如某些嵌入式设备或推理应用。近似实现通常使用查表法或多项式逼近等方法。
- **实现思路:**
 1. 遍历 IR，识别目标非线性激活函数的调用。
 2. 根据预定义的规则或配置，选择对应的快速近似函数实现。

3. 用快速近似函数的调用替换原有的激活函数调用。
4. 近似函数本身可能由一系列基本算术操作或查表操作构成。

DynamicToStatic: 查找并使动态操作静态化。

- **详细解释:** 此 Pass 尝试将计算图中的动态操作（其行为或形状依赖于运行时才能确定的值）转换为静态操作。如果发现动态操作的输入实际上是常量，Pass 会用对应的静态操作替换它们，并重新进行类型推断和常量折叠。这个过程会重复进行，直到图不再变化或达到迭代上限，从而尽可能地消除动态性，便于后续的静态分析和优化。
- **实现思路:**
 1. 遍历计算图，识别动态操作节点（例如，输入形状为动态的 Resize、Slice 等）。
 2. 分析动态操作的输入，检查它们是否是常量或来源于常量计算。
3. 如果动态输入是常量，计算出静态结果（例如，确定 Resize 后的静态形状）。
 3. 用对应的静态操作节点替换原有的动态操作节点，并使用静态结果。
 4. 替换后，可能需要重新运行类型推断和常量折叠 Pass，以利用新的静态信息。
 5. 重复以上步骤直到图稳定。

InferType: 推断表达式的类型。

- **详细解释:** 类型推断是编译器中的一个关键步骤，它分析程序中表达式的结构和操作的语义，确定每个表达式的数据类型（如 Tensor 的形状、数据类型）。此 Pass 会遍历 IR，为每个表达式填充明确的类型信息，并检查类型的一致性，确保程序的类型是合法的。类型信息对于后续的内存分配、算子选择和代码生成至关重要。
- **实现思路:**
 1. 遍历 IR 表达式树，通常采用后序遍历。
2. 对于叶子节点（如 Constant、Var），其类型已知。
 2. 对于复合表达式（如 Call、TupleGetItem），根据操作的类型签名和输入表达式的类型，推断出当前表达式的类型。
 3. 在推断过程中检查类型错误或不一致。
 4. 将推断出的类型信息记录在 IR 节点的相应字段中。

EliminateCommonSubexpr: 搜索并消除公共子表达式。

- **详细解释:** 公共子表达式消除 (CSE) 是一种经典的优化技术，它识别计算图中多次出现且计算结果相同的表达式。这个 Pass 会计算这些表达式一次，将结果存储在一个临时变量或共

享节点中，然后在所有出现该公共子表达式的地方引用这个临时变量或共享节点，从而避免重复计算，提高效率。

- **实现思路:**

1. 遍历 IR，识别具有相同结构和输入的表达式。这通常需要对表达式进行规范化表示和哈希计算来判断等价性。
2. 使用一个数据结构（如哈希表）记录已经遇到的表达式及其对应的结果（通常是一个 `Var`）。
3. 当遇到一个与已记录表达式等价的表达式时，如果其生命周期允许共享，则用已记录表达式的结果 `Var` 替换当前表达式。
4. 需要进行数据流分析，确保替换的安全性，即在替换点 `Var` 的值是有效的。

CombineParallelConv2D: 合并并行的 2D 卷积操作。

- **详细解释:** 当计算图中存在多个输入相同或相关的 2D 卷积操作并行执行时（例如，在一个 Split 或 Tuple 后），这个 Pass 会尝试将这些并行的卷积合并成一个更大的、具有更多输出通道的 2D 卷积操作。如果合并后的卷积核和偏置等参数可以有效地构造或重排，这种合并可以提高硬件利用率，减少访存和计算开销。

- **实现思路:**

1. 遍历 IR，识别输入相同或兼容的并行 `nn.conv2d` Call 节点。
2. 检查并行卷积的属性（如输入形状、步长、填充、分组等）是否兼容，可以合并。
3. 如果兼容，计算合并后卷积的输出通道数和新的卷积核、偏置等参数。
3. 创建一个新的 `nn.conv2d` Call 节点，使用合并后的参数。
4. 在原图中，用对新合并卷积的调用替换原并行的卷积调用。
5. 需要处理合并后输出的拆解，以匹配原并行的输出结构。
6. 合并的条件是并行分支数不小于 `min_num_branches`。

CombineParallelDense: 合并并行的 Dense (全连接) 操作。

- **详细解释:** 类似于 `CombineParallelConv2D`，当存在多个并行的 Dense (全连接) 操作时，此 Pass 会尝试将它们合并成一个更大的 Dense 操作，或者一个批处理矩阵乘法 (`batch_matmul`)。如果合并后的参数可以构造，这可以提高计算效率。 `to_batch_matmul` 参数控制是合并成一个更大的 Dense 还是一个 `batch_matmul`。

- **实现思路:**

1. 遍历 IR，识别输入相同或兼容的并行 `nn.dense` Call 节点。
2. 检查并行 Dense 操作的属性是否兼容。

3. 如果兼容，计算合并后操作的参数（如新的权重矩阵）。
4. 如果 `to_batch_matmul` 为 true，创建新的 `batch_matmul` Call 节点。否则，创建新的 `nn.dense` Call 节点。
5. 在原图中，用对新合并操作的调用替换原并行的 Dense 调用。
5. 需要处理合并后输出的拆解。
6. 合并的条件是并行分支数不小于 `min_num_branches`。

CombineParallelBatchMatmul: 合并并行的批处理矩阵乘法操作。

- **详细解释:** 当存在多个输入相同或相关的批处理矩阵乘法 (`batch_matmul`) 操作并行执行时，此 Pass 会尝试将它们合并成一个更大的批处理矩阵乘法操作。这有助于更好地利用硬件的批处理能力，提高计算效率。
- **实现思路:**
 1. 遍历 IR，识别输入相同或兼容的并行 `batch_matmul` Call 节点。
 2. 检查并行 `batch_matmul` 操作的属性是否兼容（如批次维度、矩阵形状）。
 3. 如果兼容，计算合并后 `batch_matmul` 的参数。
 4. 创建一个新的、具有更大批次维度的 `batch_matmul` Call 节点。
 5. 在原图中，用对新合并 `batch_matmul` 的调用替换原并行的调用。
 6. 需要处理合并后输出的拆解。
 7. 合并的条件是并行分支数不小于 `min_num_branches`。

BackwardFoldScaleAxis: 将轴缩放向后折叠到 conv/dense 算子的权重中。

- **详细解释:** 这个 Pass 会识别那些紧跟在卷积 (Conv) 或全连接 (Dense) 操作之后的轴缩放操作（例如，乘以一个向量或广播相乘），并尝试将这些缩放操作的计算逻辑融合（折叠）到前面的 Conv 或 Dense 算子的权重参数中。通过修改权重，可以直接在 Conv/Dense 计算中完成缩放，从而消除独立的缩放操作，减少计算量和内存访问。
- **实现思路:**
 1. 遍历 IR，识别 Conv 或 Dense Call 节点后紧跟着轴缩放操作（如 `multiply`、`broadcast_to` 后接 `multiply`）。
 2. 提取缩放操作的参数（如缩放因子向量）。
 3. 分析 Conv/Dense 的权重参数，判断是否可以将缩放操作的计算逻辑应用到权重上（例如，将 Conv 权重的对应通道乘以缩放因子）。
 4. 如果可以安全地折叠，计算新的权重参数。

5. 创建一个新的 Conv 或 Dense Call 节点，使用新的权重参数，并移除原有的缩放操作节点。

ForwardFoldScaleAxis: 将轴缩放向前折叠到 conv/dense 算子的权重中。

- **详细解释:** 与 BackwardFoldScaleAxis 类似，这个 Pass 识别在卷积 (Conv) 或全连接 (Dense) 操作之前的轴缩放操作，并尝试将其计算逻辑向前折叠到 Conv 或 Dense 算子的权重参数中。通过修改权重，可以在 Conv/Dense 计算前完成缩放，消除独立的缩放操作。
- **实现思路:**
 1. 遍历 IR，识别轴缩放操作后紧跟着 Conv 或 Dense Call 节点。
 2. 提取缩放操作的参数。
 3. 分析 Conv/Dense 的权重参数，判断是否可以将缩放操作的计算逻辑应用到权重上 (例如，将 Conv 权重的对应通道乘以缩放因子)。
 4. 如果可以安全地折叠，计算新的权重参数。
 5. 创建一个新的 Conv 或 Dense Call 节点，使用新的权重参数，并移除原有的缩放操作节点。

FoldScaleAxis: 顺序执行 ForwardFoldScaleAxis 和 BackwardFoldScaleAxis Pass。

- **详细解释:** 这是一个复合 Pass，它依次调用 ForwardFoldScaleAxis 和 BackwardFoldScaleAxis 这两个 Pass。通过顺序执行这两个 Pass，可以尝试将 Conv 或 Dense 操作之前和之后的轴缩放操作都折叠到权重中，从而更全面地应用缩放折叠优化。
- **实现思路:**
 1. 创建一个顺序 Pass 组合。
 2. 将 ForwardFoldScaleAxis Pass 添加到组合中。
 3. 将 BackwardFoldScaleAxis Pass 添加到组合中。
 4. 执行这个组合 Pass，它将依次运行包含的子 Pass。

CanonicalizeOps: 将一些算子规范化为简化的算子。

- **详细解释:** 这个 Pass 会将计算图中某些具有多种表示方式的操作规范化为一种标准的、简化的形式。例如，bias_add 操作 (将偏置向量加到张量的特定轴上) 可以规范化为 expand_dims (扩展偏置向量的维度) 后接 broadcast_add (广播相加)。这种规范化有助于后续的 Pass 更容易识别和处理这些模式，例如进行融合。
- **实现思路:**
 1. 遍历 IR，识别需要规范化的操作节点 (如 bias_add)。

2. 根据预定义的规范化规则，生成等价的简化操作序列（如 `expand_dims` -> `broadcast_add`）。
3. 用生成的简化操作序列替换原有的规范化操作节点。
4. 确保规范化过程不改变程序的语义。

AlterOpLayout: 改变算子的布局或用其他表达式替换原始算子.

- **详细解释:** 此 Pass 根据目标硬件的要求或性能考虑，改变计算图中算子（Operator）的输入/输出数据布局（例如，将 NCHW 布局转换为 NHWC 布局）。它也可以选择用其他等效的表达式序列替换原始的算子实现，以适配特定的后端或优化。这是实现硬件特定优化的重要步骤。
- **实现思路:**
 1. 遍历 IR，识别需要改变布局或替换实现的算子 Call 节点。
 2. 根据目标后端或配置，查找对应的布局转换规则或算子替换规则。
 3. **改变布局:** 如果需要改变布局，在算子 Call 节点前后插入布局转换操作（如 `layout_transform`），并修改 Call 节点本身的输入/输出布局属性。
 4. **替换算子:** 如果需要替换算子，生成实现相同计算逻辑的另一个表达式序列，并用该序列替换原有的算子 Call 节点。
 5. 需要确保布局转换和算子替换的正确性。

AutoSchedulerLayoutRewrite: 根据 auto-scheduler 创建的瓦片结构进行布局重写。

- **详细解释:** Auto-scheduler 是 TVM 的一个组件，用于自动探索和生成高性能的计算核函数。它会决定循环的瓦片（tiling）策略，这涉及到数据在缓存和寄存器中的布局。这个 Pass 会根据 auto-scheduler 决定的瓦片结构和数据布局，重写计算图中的数据布局和访问模式，以匹配生成的计算核的要求，确保数据以最优的方式被访存和使用。
- **实现思路:**
 1. 获取 auto-scheduler 生成的调度信息，包括瓦片大小和数据布局。
 2. 遍历 IR，识别与 auto-scheduler 调度相关的计算区域。
 3. 根据调度信息，在 IR 中插入数据重排、共享内存分配、数据拷贝等操作，将数据组织成 auto-scheduler 期望的布局。
 4. 修改计算区域内的访存指令，使其使用重写后的数据布局和索引计算。

MetaScheduleLayoutRewrite: 根据 meta-schedule 创建的瓦片结构进行布局重写。

- **详细解释:** Meta-schedule 是 TVM 的另一个自动调优组件，它也生成优化的调度策略，包括瓦片结构和数据布局。这个 Pass 的功能类似于 `AutoSchedulerLayoutRewrite`，但它是根据 meta-schedule 生成的调度信息来重写计算图中的数据布局 and 访问模式，以适配 meta-schedule 生成的计算核。
- **实现思路:**
 1. 获取 meta-schedule 生成的调度信息，包括瓦片大小和数据布局。
 2. 遍历 IR，识别与 meta-schedule 调度相关的计算区域。
 3. 根据调度信息，在 IR 中插入数据重排、共享内存分配、数据拷贝等操作，将数据组织成 meta-schedule 期望的布局。
 4. 修改计算区域内的访存指令，使其使用重写后的数据布局 and 索引计算。

ConvertLayout: 将 expr 转换为目标布局，以使大多数操作的输入数据布局改变为目标布局。

- **详细解释:** 此 Pass 的主要目标是将整个计算图或其中的大部分，转换为使用指定的目标数据布局。它会分析计算图，并在必要的地方插入布局转换操作，使得大部分算子能够以目标布局处理数据。理想情况下，布局转换只发生在图的入口和出口处。这对于只支持或偏好特定数据布局的硬件后端非常有用。
- **实现思路:**
 1. 接收一个映射，指定哪些操作类型需要转换为哪些目标布局 (`desired_layouts`)。
 2. 遍历计算图，从图的输入开始，根据 `desired_layouts` 确定期望的数据布局。
 3. 使用布局传播和推断算法 (`InferCorrectLayout` 基础设施) 来确定图中每个表达式的布局。
 4. 在布局不匹配的地方 (即一个操作需要某种输入布局，但其前一个操作输出的是另一种布局)，插入布局转换操作 (`layout_transform`)。
 5. 修改相关操作的输入，使其接收布局转换后的结果。
 6. Legalize: 使用另一个表达式合法化 `expr`。
- **详细解释:** 这个 Pass 会遍历计算图，并根据预定义的"合法化" (Legalize) 规则，用另一个表达式序列替换某些操作或表达式，使其符合目标后端或特定约束的要求。例如，一个后端可能不支持某种复杂操作，但支持一系列基本操作的组合。Legalize Pass 就会用这系列基本操作替换原有的复杂操作。`legalize_map_attr_name` 参数用于指定查找合法化规则的属性名称。

- **实现思路:**

1. 遍历 IR，识别具有指定合法化属性 (`legalize_map_attr_name`) 的操作或表达式。
2. 对于每个识别出的节点，查找与其关联的合法化规则函数。
3. 调用合法化规则函数，生成等价的替换表达式序列。
4. 用生成的表达式序列替换原有的节点。
5. 确保替换后的计算逻辑与原节点一致。

CanonicalizeCast: 规范化 cast 表达式以提高算子融合效率。

- **详细解释:** Cast 操作用于在不同数据类型之间进行转换（例如从 float32 转换为 float16）。这个 Pass 会对 Cast 表达式进行规范化处理，例如合并连续的 Cast 操作、将 Cast 下推或上提，以便更好地暴露算子融合的机会。规范化的 Cast 表达式更容易被融合 Pass 识别和处理，从而提高融合的成功率和效率。

- **实现思路:**

1. 遍历 IR，识别 Cast 操作。
2. 识别连续的 Cast 操作，将其合并为一个 Cast（如果可能）。
3. 尝试将 Cast 操作移动到计算图中的不同位置，例如推下到操作输入，或上提到操作输出，同时维护语义正确性。
4. 应用预定义的规范化规则，使 Cast 表达式的结构更标准，便于后续分析和融合。

EtaExpand: 在构造函数或绑定到函数的全局变量上添加抽象。

- **详细解释:** Eta-expansion 是一种函数式编程概念，此 Pass 将构造函数或绑定到函数的全局变量转换为一个显式的 Lambda 抽象（函数）。例如，一个直接引用全局函数的表达式 `func` 可能会被转换为 `fn (%x) => func(%x)`。这使得函数引用更显式，有助于统一处理函数和值，并可能为后续的函数式变换和优化创造机会。

- **实现思路:**

1. 遍历 IR，识别对构造函数或绑定到函数的全局变量的直接引用。
2. 根据引用对象的类型和参数，创建一个新的 Lambda 表达式（函数），该 Lambda 接受与原对象相同的参数，并在函数体内调用原对象。
3. 用新创建的 Lambda 表达式替换原有的引用。
4. `expand_constructor` 和 `expand_global_var` 参数控制对哪种类型的对象进行 Eta-expansion。

PartitionGraph: 将 Relay 程序划分为可以在不同后端上执行的区域。

- **详细解释:** 这个 Pass 负责将整个计算图根据操作的设备亲和性或用户指定的划分策略，分割成多个子图或区域，每个区域可以被编译并在特定的硬件后端（如 CPU、GPU、DSP）上执行。这对于异构计算场景至关重要，它使得程序的不同部分可以在最适合的硬件上运行。
- **实现思路:**
 1. 遍历计算图，分析操作的属性（如设备注解、编译器属性）和连接关系。
 2. 根据划分规则（例如，属于同一后端编译器的操作划分为一个区域），识别可划分的子图。
 3. 将每个子图封装成独立的函数或模块。
 4. 在主图中，用对子图函数的调用替换原有的子图部分。
 5. 需要处理子图之间的输入/输出数据传递，可能需要插入设备间的数据拷贝操作。

Inline: 内联给定 Relay IRModule 中标记为 `inline` 的全局函数。

- **详细解释:** 此 Pass 会识别在 Relay IRModule 中被明确标记为可以内联的全局函数。它会将这些函数的调用点替换为被调用函数的实际代码体。内联可以消除函数调用带来的开销（如栈操作、跳转），增加指令级并行性，并且可能暴露更多的跨过程优化机会（例如，被调函数体内的代码可以与调用点的代码一起进行常量折叠、死代码消除等）。
- **实现思路:**
 1. 遍历 IRModule，识别具有 `inline` 属性的全局函数。
 2. 遍历整个 IRModule，找到对这些可内联函数的调用点。
 3. 在每个调用点，复制被调用函数的函数体。
 4. 处理参数传递和返回值，确保内联后的代码逻辑与原函数调用一致。
 5. 用复制的函数体替换原有的函数调用节点。
 6. 如果一个函数在内联后不再被其他地方调用，可以考虑移除其定义。

RemoveUnusedFunctions: 移除 Relay IRModule 中未使用的函数。

- **详细解释:** 这个 Pass 会识别并删除 Relay IRModule 中那些从指定的入口函数（`entry_functions`）开始无法访问到的函数。这些函数可能是旧代码、实验性函数或者在重构后不再需要的部分。移除未使用的函数可以减小编译后的模型体积。
- **实现思路:**
 1. 从 `entry_functions` 列表开始，进行可达性分析（例如，使用图遍历算法）。
 2. 标记所有从入口函数可达的函数。

3. 遍历 IRModule 中的所有函数定义。
4. 删除所有未被标记为可达的函数。

SimplifyExpr: 简化 Relay 表达式。

- **详细解释:** 这是一个通用的表达式简化 Pass，它会应用一系列简化规则来简化计算图中的表达式。这可能包括算术和逻辑表达式简化、常量折叠、简单的代数变换等。目标是减少表达式的复杂性，消除冗余计算，使 IR 更简洁高效，为后续的优化创造条件。
- **实现思路:**
 1. 结合多种表达式简化技术（如算术简化、逻辑简化、常量折叠）。
 2. 反复遍历 IR，应用简化规则，直到不再发生变化。
 3. 需要实现各种操作的简化规则（例如， $x * 1$ 简化为 x ， $x + 0$ 简化为 x ）。

SimplifyExprPostAlterOp: 在 AlterOpLayout 后运行的简化版 SimplifyExpr。

- **详细解释:** 这个 Pass 是 `SimplifyExpr` 的一个简化版本，它专门设计用于在 `AlterOpLayout` Pass 之后运行。`AlterOpLayout` 可能会引入一些新的简单的表达式（例如布局转换后的 Reshape、Transpose），这个 Pass 会对这些新生成的表达式进行清理和简化，但通常不会执行像完整的 `SimplifyExpr` 那样激进或复杂的优化，以避免干扰后续的低级优化。
- **实现思路:**
 1. 实现一个包含部分简化规则的简化器。
 2. 遍历 IR，应用这些简化规则。
 3. 规则集通常比完整的 `SimplifyExpr` 要小且针对性更强，主要处理布局重写可能引入的模式。

RelayToTIRTargetHook: 在 TargetKinds 上运行注册在 "RelayToTIR" 属性下的自定义 Pass。

- **详细解释:** 这个 Pass 是一种扩展机制，它会查找那些带有特定"编译器" (Compiler) 属性的函数。如果该属性值对应的 TargetKind 注册了"RelayToTIR"属性并关联了一个自定义 Pass，那么这个自定义 Pass 就会被执行。这个自定义 Pass 通常负责将 Relay IR 的特定部分（通常是标记给特定后端的计算）转换为更低级别的 TIR (Tensor IR) 或直接生成后端可执行模块。它允许开发者为特定的硬件后端集成自定义的编译流程。
- **实现思路:**

1. 遍历 IRModule，识别具有 `Compiler` 属性的函数。
2. 根据 `Compiler` 属性值查找对应的 TargetKind。
3. 检查 TargetKind 是否注册了 `RelayToTIR` 属性，并获取关联的自定义 Pass。
4. 如果找到自定义 Pass，则执行它，并将当前的目标信息 (`CompilationConfig`) 传递给它。
5. 自定义 Pass 负责将 Relay 函数转换为 TIR 或外部模块，并更新 IRModule。

ManifestAlloc: 一个用于显式内存分配和重写特定方言的 Pass。

- **详细解释:** 此 Pass 会分析计算图中的内存使用，并将抽象的内存需求转换为显式的内存分配操作。它会决定何时、何地分配多大的缓冲区来存储中间结果和变量。同时，它也可能负责重写或处理一些特定的"方言" (Dialects) 操作，将其转换为更标准或低级别的表示，为后续的内存优化和代码生成做准备。 `cpu_virtual_device` 参数指定了 CPU 上计算和数据使用的虚拟设备。
- **实现思路:**
 1. 遍历 IR，识别需要分配内存的中间表达式结果和变量。
 2. 进行活跃性分析，确定变量的生命周期和内存需求。
 3. 在 IR 中插入显式的内存分配节点 (如 `Alloc`) 。
 4. 修改相关的 Load/Store 操作，使其引用新分配的内存地址。
 5. 同时，实现对特定方言操作的识别和重写规则。

ManifestLifetimes: 通过在变量不再活跃时插入 kill 操作来显式化变量生命周期。

- **详细解释:** 在 `ManifestAlloc` 显式分配内存后，这个 Pass 会进一步显式化内存的生命周期管理。它会分析变量的活跃范围，并在变量最后一次使用后、内存可以被回收或重用的地方，插入 `kill` 操作。 `kill` 操作明确地标记了变量的生命周期结束点，这有助于后续的内存重用、缓冲区分配和垃圾回收优化。
- **实现思路:**
 1. 在 `ManifestAlloc` 之后运行，此时内存分配已显式。
 2. 对程序进行数据流分析，确定每个变量或内存区域的活跃范围 (从第一次使用到最后一次使用) 。
 3. 在变量的最后一次使用点之后，插入 `kill` 操作节点，标记该变量对应的内存区域可以被释放或重用。
 4. 需要确保 `kill` 操作不会过早地释放仍然活跃的内存。

PlanDevices: 使用现有的 "on_device" 和 "device_copy" CallNode 来推断每个 Relay 子表达式应该运行在哪个 VirtualDevice 上并存储结果。

- **详细解释:** 这个 Pass 基于程序中已有的设备注解（如 `on_device` CallNode 指示某个子表达式应在特定设备上执行，`device_copy` 指示数据在设备间拷贝）来推断整个计算图中每个子表达式应该在哪个虚拟设备（VirtualDevice）上执行以及其结果应该存储在哪里。推断完成后，它会通过插入新的 `on_device` 和 `device_copy` CallNode 来显式地标记这些决策，为后续的异构代码生成提供明确的设备信息。
- **实现思路:**
 1. 遍历计算图，收集现有的 `on_device` 和 `device_copy` 注解信息。
 2. 实现一个设备推断算法，根据注解、操作的默认设备（`CompilationConfig`）和连接关系，推断出图中每个节点的执行设备和结果存储设备。
 3. 遍历 IR，在确定设备边界的地方，插入新的 `on_device` CallNode 来标记执行设备。
 4. 在需要跨设备传输数据的地方，插入新的 `device_copy` CallNode。
 5. 需要处理设备约束的冲突和优先级。

FlattenAtrousConv: 平展空洞卷积，将其转换为具有修改后的"dilation"和重新计算的"padding"参数的子图。

- **详细解释:** 空洞卷积（Atrous Convolution 或 Dilated Convolution）通常可以表示为 `space_to_batch_nd` -> `conv2d` -> `batch_to_space_nd` 这一系列操作。这个 Pass 会识别这种模式，并将其重写为一个等价的子图，其中包含一个普通的 `conv2d` 操作，但其 `dilation` 和 `padding` 参数会被重新计算以模拟空洞卷积的效果。这样做有助于简化图结构，并可能为融合或其他 Conv 相关优化创造机会。
- **实现思路:**
 1. 遍历 IR，识别 `space_to_batch_nd` 后接 `conv2d` 再接 `batch_to_space_nd` 的模式。
 2. 提取原 `conv2d` 的参数（如权重、步长、填充）以及 `space_to_batch_nd` 和 `batch_to_space_nd` 的参数（如 `block_shape`）。
 3. 根据空洞卷积的数学定义和参数关系，计算等效普通 `conv2d` 的新 `dilation` 和 `padding` 参数。
 4. 创建一个新的 `conv2d` Call 节点，使用原始权重、步长和计算出的新 `dilation` 和 `padding`。
 5. 用新的 `conv2d` Call 节点替换原有的三个操作序列。

AnnotateUsedMemory: 通过分析每个原始函数调用点的输入/输出张量的活跃性，标注每个调用所需的最小内存。

- **详细解释:** 这个 Pass 会分析计算图中每个原始函数 (PrimFunc) 调用点的输入和输出张量的活跃范围。通过跟踪这些张量的生命周期，它可以计算出在每个调用点所需占用的最小内存总量。这个信息会被作为注解 (`used_memory`) 添加到相应的函数上，通常以字节数列表的形式表示每个调用点的内存需求。此外，包含这些调用的函数也会被标注一个 `io_used_memory`，表示输入输出张量总共所需的内存。这个 Pass 不支持动态形状。
- **实现思路:**
 1. 遍历计算图，识别原始函数 (PrimFunc) 的调用点。
 2. 对于每个调用点，进行数据流分析或活跃性分析，确定其输入和输出张量的活跃范围。
 3. 根据张量的形状和数据类型，计算每个活跃张量所需的内存大小。
 4. 对所有活跃的输入输出张量所需的内存求和，得到该调用点的最小内存需求。
 5. 将这些内存需求作为 `used_memory` 注解添加到被调函数上，并将输入输出总内存作为 `io_used_memory` 添加到调用者函数上。
 6. 需要处理 Tuple 等复合结构。

CapturePostDfsIndexInSpans: 在表达式节点的 span 中捕获后序 DFS 索引和支配者后序 DFS 索引。

- **详细解释:** 这个 Pass 用于调试和分析目的。它会遍历计算图中的表达式节点（大多数），并计算它们的后序深度优先搜索 (Post-DFS) 索引以及其支配者节点 (Dominator) 的后序 DFS 索引。这些索引信息会被添加到节点的 `span` 属性中，格式通常为 "index::"。这有助于在打印的 IR 中快速定位子表达式，并且这些索引在 Collage 等依赖图结构的工具中也很实用。Op 和 Constructor 节点通常不会被修改。
- **实现思路:**
 1. 对计算图进行后序深度优先搜索，计算每个可标注节点的 Post-DFS 索引。
 2. 计算计算图的支配者树。
 3. 对支配者树进行后序深度优先搜索，计算每个节点的支配者 Post-DFS 索引。
 4. 遍历原始计算图，对于可标注的节点，将计算出的两个索引格式化为字符串，添加到其 `span` 属性中。

AnnotateMemoryScope: 调用设备相关的内存作用域分析 Pass，收集期望的 `expr->memory_scope` 映射并用所需的 `memory_scope` 标注表达式的 `VirtualDevice`。

- **详细解释:** 这个 Pass 负责确定计算图中每个表达式的结果应该存储在哪个内存作用域 (Memory Scope) 中，例如全局内存、共享内存、局部寄存器等。它会调用特定于目标设备或后端的内存作用域分析工具，根据硬件特性、访问模式和用户注解，推断出最优的内存作用域。然后，它会更新表达式关联的 `VirtualDevice` 的 `memory_scope` 属性，明确指定结果的存储位置。
- **实现思路:**
 1. 调用或集成设备相关的内存作用域分析 Pass。
 2. 该分析 Pass 遍历 IR，根据设备模型、操作类型、访问模式和已有的内存注解，生成一个映射，指示每个表达式结果推荐存储的内存作用域。
 3. `AnnotateMemoryScope` Pass 获取这个映射。
 4. 遍历 IR，对于每个表达式，根据映射找到其推荐的内存作用域。
 5. 更新该表达式关联的 `VirtualDevice` 的 `memory_scope` 字段。

RemoveStandaloneReshapes: 降低图后移除非融合的 reshape 操作。

- **详细解释:** 在计算图被降低 (Lowering) 到较低级别的表示之后，一些 Reshape 操作可能不再是必要的计算，而更多是符号性的形状转换。特别是在某些后端不关心具体的张量形状，只需要缓冲区地址的情况下。这个 Pass 会移除那些不再影响计算逻辑或控制流的独立的 (即未被融合到其他操作中的) Reshape 操作。移除 Reshape 可以减小代码体积，减少缓冲区分配，并为后端提供更多融合机会，从而提升推理性能。需要注意的是，在此 Pass 后不能再调用 `InferType`，因为它改变了图的连接结构。
- **实现思路:**
 1. 在图降低到一定程度后运行此 Pass。
 2. 遍历 IR，识别 `reshape` Call 节点。
 3. 分析 `reshape` 节点的使用情况，判断它是否是独立的 (即其输出没有被融合到其他操作中，或者仅仅用于形状传递)。
 4. 如果 `reshape` 是独立的且可以安全移除 (即移除后不影响程序逻辑或正确性)，则将 `reshape` 的输入直接连接到其所有使用点，并移除 `reshape` 节点。